# Kinefly2: Daily Checklist

Turning everything on before an experiment, changing things during the day, and shutting down.

For people already using Kinefly2. New users: start with *Kinefly2: Getting Started*. Code version 9ccc802, October 2026.

## A Before the experiment

1. **Power on** the rig PC, the camera(s) and the IR backlight. **After a reboot**, run this unless it has been made permanent: `sudo sh -c 'echo 1000 > /sys/module/usbcore/parameters/usbfs_memory_mb'`
2. **Open a terminal:** `cd Kinefly2` then `conda activate kinefly2`.
3. **Check the template** `experiments/<project>.yaml`: `data_root`, `genotype`, `record_video`. Your script sets the fly ID and protocol for each recording.
4. **Start Kinefly2** and leave it running all day:

   ```
   $ python -m kinefly2.apps.run_rig config/rig.yaml --exp experiments/<project>.yaml --scope
   ```

   **Bottom camera only:** set `front: enabled: false` in `config/rig.yaml`, or add `--cameras bottom` to the command. The banner should print `Kinefly2 up: cameras bottom, primary bottom`.
5. **Health check:** `python -m kinefly2.apps.ctl status`. Each camera's `fps` should match the banner's `rate:` line, with `down: false`.
6. **Background:** with **no fly in view**, click `saveBG`. Do this daily, and again whenever the light, camera or holder moves.
7. **Mount the fly**, then click `autoplace`. Check three things:
   - the wing hinges sit on the wing bases;
   - the head landmarks are in front of the neck;
   - the overlay shows **no warnings**.

   Nudge handles as needed.
8. **Let the fly settle**, then click `zero`. This sets the head's zero posture.
9. **Watch the Scope for a minute.** The traces should move with the fly, without long NaN gaps.

## B Run the experiment

```
import time
from kinefly2.client import Kinefly2Client
k = Kinefly2Client('localhost')            # rig IP if the script runs elsewhere

# the recording stops when this block ends, even after an error
with k.recording(fly_id='018', protocol='YAMV', notes=''):
    k.stim(0)                              # baseline label (it persists between recordings)
    for trial, (pid, dur) in enumerate(PATTERNS):
        arena.show_pattern(pid)            # your arena's API call (placeholder)
        k.stim(pid)                        # IMMEDIATELY after: frames now labelled pid
        k.event({'trial': trial, 'event': 'stim_on', 'pattern': pid})
        time.sleep(dur)
        arena.stop(); k.stim(0)
```

Without a script, use the `record` button, or `python -m kinefly2.apps.ctl start --fly 018 --protocol YAMV` and then `... ctl stop`. Data goes to `<data_root>/<date>/<date>_<genotype>_<fly>_<protocol>/`.

## C Changing things during the day

| To change | Do this | Takes effect |
|---|---|---|
| **Fly, protocol, notes, genotype** | Pass them to `k.recording(...)` / `k.start(...)`, or use `ctl start --fly … --protocol … --genotype … --notes …` | Next recording |
| **Video on / off** | `k.start(..., record_video=False)` or `ctl start --no-video` | Next recording |
| **Stimuli** (patterns, order, durations) | Edit the pattern list in **your script**. Kinefly2 needs no change; just keep calling `k.stim(pid)` right after each arena call. | Next run of the script |
| **Regions**: hinges, windows, landmarks, track / subtract boxes | Drag or click in the camera window. Saved to the camera YAML when you release the mouse. Change them **between** recordings, not during one. | Immediately |
| **Head zero** | The `zero` button, or `k.zero()`. It is logged as an event if a recording is running. | Immediately |
| **New fly** | New `fly_id`, then `autoplace`, check the overlay, `zero`. Re-take `saveBG` only if the light, camera or holder moved. | Next recording |
| **Settings in files** (tracker, camera, template) | Edit `tracker:` / `camera:` in `config/camera_bottom.yaml`, or `experiments/<project>.yaml`. A restart keeps your regions, background and head templates. | After restarting `run_rig` |
| Two cameras: `lever_px` | Run `calibrate_fly <recording> --write …yaml` and restart, or pass `k.start(..., lever_px={'front': 48.2})`. | Next recording |

## D End of the day

1. **Let the script finish.** Its `with` block stops the last recording.
2. **Quit:** `q` or `exit` in a camera window (Ctrl+C with `--no-gui`). An open recording is closed cleanly.
3. **Check and back up:** `python -m kinefly2.apps.scope <folder>/*_flystate.h5`, back up `<data_root>/<date>/`, then switch off the backlight.

## E Quick fixes

| Symptom | Fix |
|---|---|
| `no reply from Kinefly2 at …` | `run_rig` is not running, the host/IP is wrong, or a firewall blocks port 5558. |
| `already recording …` | The previous recording is still open. Run `k.stop()` or `ctl stop`. |
| Camera `down` / "reconnecting…" | It retries every 2 s. Check the USB cable; after a reboot run the `usbfs` line (A1). |
| `list_cameras`: "in use or unavailable" | Close SpinView, or any other Kinefly2 holding the camera. |
| `reaches only … fps` at start | Make `roi` smaller, or `exposure_us` shorter than one frame. |
| `REC stop … LOG DROPPED` | The disk is too slow. Record to a local SSD, or turn video off. |
| Traces noisy, jumping or NaN | Fix the overlay warning (Getting Started §5); re-take `saveBG` with no fly. |
| **STALE** badge (two cameras) | The front camera is falling behind. Check its cable and the trigger wiring. |